Формалне методе
(Увод у TLA+)

Филип Марић
Александар Ивановић

7. април 2026.

Садржај

1 Увод - Шта је TLA? 4

1.1 Инсталација . . . 4

1.1.1 Кораци инсталације (TLA+ Toolbox) . . . 4

1.1.2 Кораци инсталације (VS Code editor) . . . 4

2 Основни појмови 9

2.1 Основни делови tla спецификације . . . 9

2.2 Конфигурациони фајл . . . 11

3 Неки основни примери 12

3.1 Прекидач за светло . . . 12

3.2 Брава на вратима . . . 13

3.3 Узајамно искључење . . . 14

4 Напредни примери 17

4.1 Outbox pattern . . . 17

5 Доказивање теорема у TLA+ 24

5.1 Структура доказа . . . 24

5.2 Пример: бројач са инваријантом . . . 24

6 Преглед TLA+ оператора 27

6.1 Логички оператори . . . 27

6.2 Релацијски и аритметички оператори . . . 27

6.3 Оператори стања и прелаза . . . 27

6.4 Теоријско-скуповни оператори . . . 27

6.5 Оператори над функцијама и торкама . . . 28

6.6 Квантификатори и `LET/IN` . . . 28

6.7 Кључне речи модула . . . 28

6.8 Оператор опсега . . . 28

6.9 Записи . . . 28

6.10 Секвенце (`EXTENDS Sequences`) . . . 29

6.11 Оператор `CHOOSE` . . . 29

6.12 Израз `CASE` . . . 29

6.13 Оператори правичности (Fairness) . . . 29

6.14 Оператор `ENABLED` . . . 30

6.15 Модул `FinSets` (коначни скупови) . . . 30

6.16 Конфигурациони фајл (`.cfg`) . . . 30

Предговор

Ова скрипта је настала у оквиру предмета **Формалне методе - напредни концепти** на докторским студијама из информатике на Математичком факултету универзитета у Београду. У овом тренутку ово је текст у раној фази писања и садржи грешке које ће аутори исправљати током рада на скрипти. Идеја је свакако да и будуће генерације студената раде на њој и обогате садржај. Скрипта се бави језиком **TLA+** који служи за проверу дизајна система и који су већ користили и користе највеће светске компаније попут: Мајкрософта, Амазона, MongoDB, и Cockroach Labs. Идеја је да скрипта омогући лако инсталирање тла система и да кроз практичне примере објасни основне концепте, како и самог језика, тако и кроз примере неке свакодневне програмерске праксе из искуства аутора. Такође желимо да повежемо и укажемо на везе математике и рачунарства као једне од њених најпрактичнијих грана без набацивања превише теорије.

1 Увод - Шта је TLA?

TLA је језик који за писање и проверавање спецификација или дизајнирање система. Када имамо нашу спецификацију тада га можемо користити за тестирање грешака у дизајну и пре него што почнемо са самом имплементацијом система. Развијен је од стране Леслија Лампорта, који је такође осмислио језик LaTeXшто је скраћеница од Лампортов TeX. TLA је скраћеница за Temportal Logic of Actions (Темпорале Логике Акција).

1.1 Инсталација

Да бисмо почели да пишемо и проверавамо TLA+ спецификације, потребно је инсталирати следеће алате:

- **Java** – TLA+ алати захтевају Java Runtime Environment (JRE 8 или новији). Може се преузети са `https://www.java.com`.
- **TLA+ Toolbox** – званични IDE за TLA+, доступан на `https://github.com/tlaplus/tlaplus/releases`. Садржи TLC model checker и TLAPS (TLA+ Proof System).
- **VS Code додатак (алтернатива)** – за кориснике Visual Studio Code, постоји додатак `TLA+ by alygin` доступан у VS Code Marketplace-у.

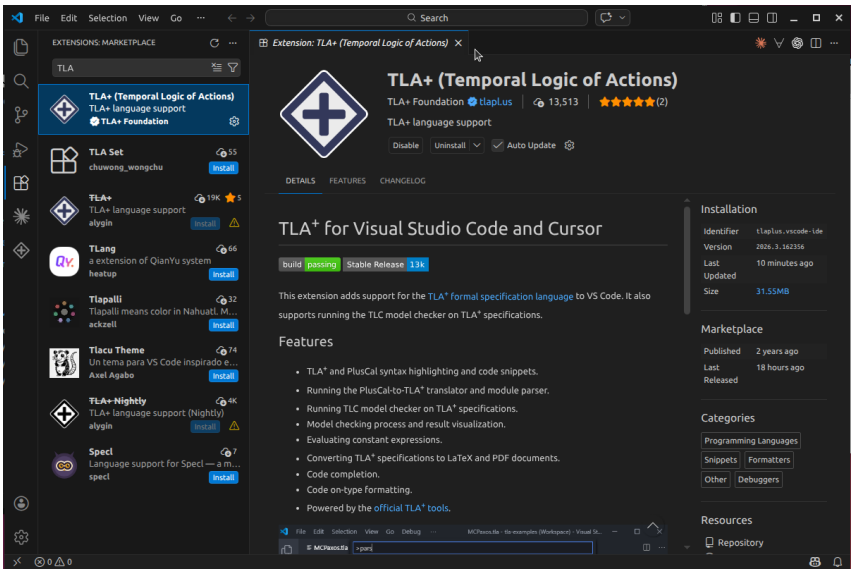
1.1.1 Кораци инсталације (TLA+ Toolbox)

1. Преузети инсталациони пакет са званичне странице за свој оперативни систем.
2. Покренути инсталацију и пратити упутства.
3. Покренути Toolbox и креирати нови пројекат: *File > Open Spec > Add New Spec*.
4. Написати прву спецификацију и покренути TLC model checker.

1.1.2 Кораци инсталације (VS Code editor)

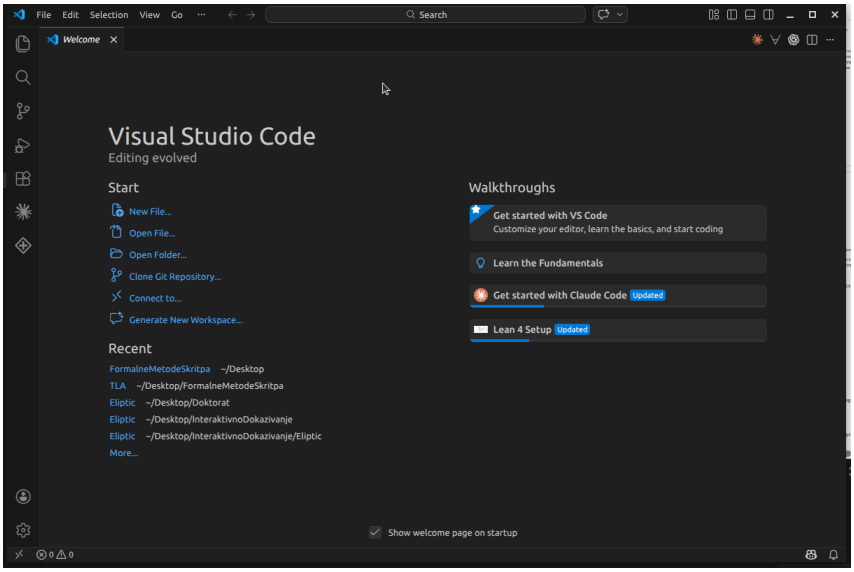
У овим материјалима користићемо VS Code editor (надље само **вс едитор**) и објаснићемо примере и инсталацију на примеру њега у оперативном систему линукс. Напомињемо да се TLA+ Toolbox више не одржава, али да се и даље може скинути и користити. Вс едитор је постао стандард за писање спецификација и TLA се може користити кроз њега као додатак. Иначе напомињемо да је вс едитор постао популаран едитор за разне програмске језике управо због модуларности коју пружа богат избор додатака.

Изаберемо иконицу са додацима где се они инсталирају као на слици 1.



Слика 1: Инсталација TLA+ додатка у VS Code

Приликом избора додатка који ћемо инсталирати у претрагу укуцамо TLA и одаберемо додак као на слици. На крају процеса едитор треба да изгледа као на слици 2.



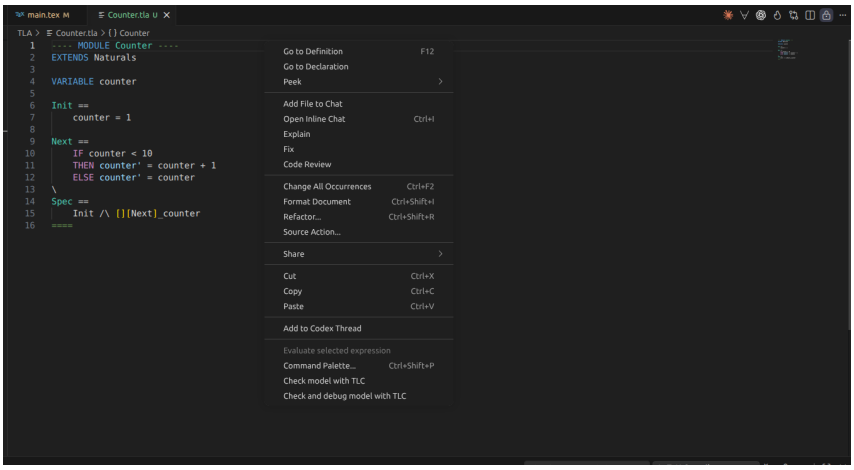
Слика 2: VS Code едитор са TLA+ додатком

Сада дајемо мали пример TLA спецификације како би тестирали дали окружење ради добро.

```
---- MODULE Counter ----
EXTENDS Naturals

VARIABLE counter

Init ==
    counter = 1
```

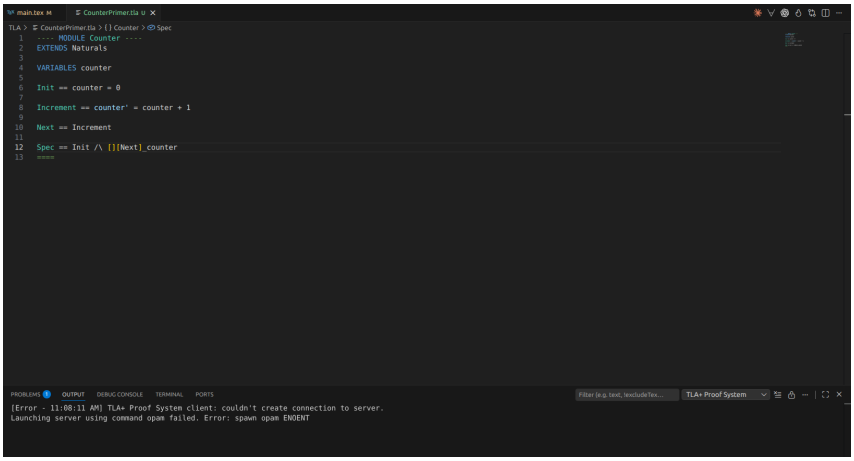
```
Next ==
    IF counter < 10
    THEN counter' = counter + 1
    ELSE counter' = counter

Spec ==
    Init /\ [][Next]_counter
====
```

У едитору ћемо направити нов текстуални фајл са наставком **.tla** и прекопирати претходни код. Пошто смо инсталирали TLA+ додатак на десни клик мишшем добијамо могућност да одаберемо **Check model with TLC** као и **Check and debug model with TLC**. **TLC** је скраћеница за **TL checker** и он нам служи да пронађе грешке у нашем дизајну система.



Слика 3: Код са TLA+ додатком

Када кликнемо да проверимо модел тада се може јавити грешака као на следећој слици.



Слика 4: Грешка при покретању провере

Грешка се јавља зато што **TLAPS - TLA Proof System** покушава да користити

програмски језик **OCaml**. Ова грешка се на линукс оперативном систему једноставно решава инсталацијом пакета **OPAM** следећим командама у терминалу:

```
sudo apt update
sudo apt install opam m4 pkg-config zlib1g-dev gawk time
```

Напомињемо да су ово команде за линукс/**Ubuntu**, ако користите неку другу дистрибуцију линукса потражите одговарајуће команде. Уз сваку tla спецификацију (.tla) фајл, такође нам је потребан и конфигурациони фајл са наставком (.cfg) који нам говори шта су почетна стања, како се прелази у следеће стање, константе итд.

```
CONSTANTS
    greeting = "Hello"

INIT Init
NEXT Next
```

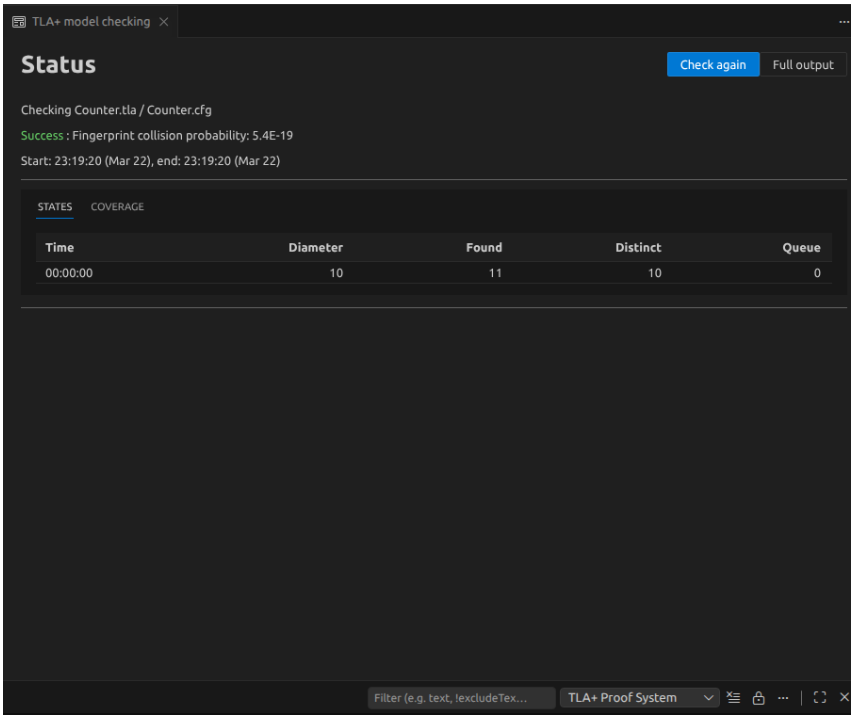
Конфигурациони фајл са претходним кодом морамо да сачувамо у исти фолдер где се налази и наша спецификација, такође ти фајлови морају имати исто име као и прва линија у спецификацији (**Counter.tla**, **Counter.cfg**).

```
---- MODULE Counter ----
```

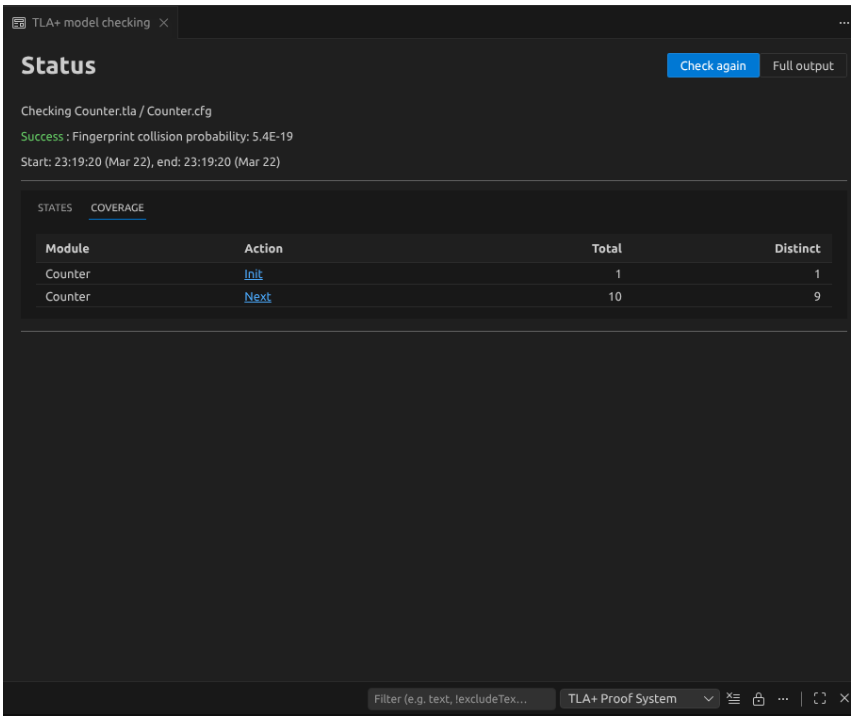
Кључне ствари које морамо да имамо за сваку спецификацију су:

1. Спецификациони фајл са наставком **.tla**.
2. Конфигурациони фајл са наставком **.cfg**.
3. Они морају да имају исто име, као и име у првој линији у спецификацији.

Пошто смо све урадили, када покренемо проверу спецификације, ако је све у реду добијамо успех као на слеће две слике.



Слика 5: Успех провера стања

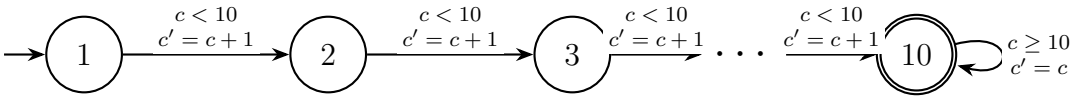


Слика 6: Успех провера покривености

Ово значи да смо успешно успели да инсталирамо tla на линуксу и да наша прва спецификација система ради без грешке. У следећем поглављу ћемо корак по корак објашњавати шта који део спецификације значи и чему служи.

2 Основни појмови

У овом поглављу ћемо да објаснимо основне елементе нашег примера из претходне секције. Концептуално идеја ТЛА+ је да систем представи као неки граф могућих стања који одређеује која су почетна, крајња стања и како се прелази у следеће стање система на основу тренутног и правила прелаза. Ово нас доста подсећа на аутомате из ранијих курсева за компајлере и лексичку анализу.



Слика 7: Граф прелаза система Counter: стања су вредности бројача (c), прелази одговарају повећавању за 1 све до вредности 10

Претходна слика представља граф нашег система из уводног примера са свим својим могућим стањима. Присећамо се да почетно стање има само улазну стрелицу, а крајње обележавамо са два концентрична круга. Као што можемо видети почетно стање нашег система је стање **1**, а крајње стање је **10**. Прелаз између стања је могућ ако променљива **counter** има вредност мању или једнаку од 10 и ако смо у стањима 1 ди 9. На основу спецификације у тла, тла систем генерише граф стања система и обилази га и покушава да нађе неке грешке: недоступа стања, бесконачне петље итд.

2.1 Основни делови tla спецификације

Да се подсетимо претходног уводног примера и конфигурационог фајла.

```
---- MODULE Counter ----
EXTENDS Naturals

VARIABLE counter

Init ==
    counter = 1

Next ==
    IF counter < 10
    THEN counter' = counter + 1
    ELSE counter' = counter

Spec ==
    Init /\ [][Next]_counter
====
```

```
CONSTANTS
    greeting = "Hello"

INIT Init
NEXT Next
```

Видимо да уводни пример у потпуности одговара графу који смо нацртали. За велике и озбиљне системе ручно проверавање стања је практично немогуће и ту нам треба

помоћ рачунара. Укратко смо представили поједностављену верзију како tla систем функционише. Сваки tla спецификација почиње са кључном речи **Module**. Симболи **- - -** су део стандардне синтаксе који означавају име и почетак спецификације. Такође $= = =$ на самом крају фајла означава крај спецификације и нема неки већи значај-

```
---- MODULE Counter ----
...
====
```

Кључна реч **EXTENDS Naturals** нам увози библиотеку која нам омогућава да радимо са стандардним аритметичким операцијама $+, \cdot, >, <$ итд.

```
---- MODULE Counter ----
EXTENDS Naturals
...
====
```

Да нисмо увезли ову библиотеку не бисмо могли да радимо са изразима попут $counter < 10$ и $counter + 1$.

VARIABLE counter нам декларише променљиву са именом *counter*.

```
---- MODULE Counter ----
...

VARIABLE counter

Init ==
    counter = 1
...
====
```

Променљиве су део стања програма и њихова вредност се мења између стања, тј. при прелазу између стања добијају вредности. У нашем случају имамо једну променљиву `counter` и њена вредност се очигледно мења при промени стања.

```
Init ==
    counter = 1
```

Почетна вреднот променљиве у почетном стању је једнака 1.

Следећи део кода нам говори како се прелази у следеће стање програма и томе служи кључна реч **Next**.

```
---- MODULE Counter ----
...
Next ==
    IF counter < 10
    THEN counter' = counter + 1
    ELSE counter' = counter
...
====
```

Структура **IF ... THEN ... ELSE** има стандардно значење као и у свим програмским језицима, ако је услов испуњен улазимо у прву грану, а у супротном у **ELSE** грану. Оно што се разликује од досадашњег искуства је апостроф над промљивом `counter'`. Апостроф овде нам служи да кажемо шта је ново стање променљиве `counter` и како

га добијамо. У овом случају ново стање променљиве је једнаком старом увећано за 1 ако је вредност из претходног стања мања од 10, а у супротоном остаје непромењена. Када име променљиве пишемо без апострофа то значи да узимамо њену вредност из претходног стања.

Последњи део спецификације нас асоцира на неки логичку конјукцију два услова. Оператор /\ има стандардно значење логичке конјукције $\wedge$, па овај део спецификације можемо тумачити да би спецификација била исправна потребно је да оба услова буду испуњена.

```
---- MODULE Counter ----
...
Spec ==
    Init /\ [][Next]_counter
====
```

Систем је свакако почео у стању **Init** и вредност променљиве јесте 1 на почетку. Други део конјукције користи важан оператор **[]**. Оператор **[]** значи да неки услов треба да буде испуњен **увек**, то значи да има временско трајање, па одатле и добијамо назив TLA - temporal logic of actions. Оператор **[Next]** нам кажемо да свако следеће стање при прелазу треба да се добија из стања **Next**, а `_counter` значи да стање може да буде непромењено при прелазу. Говорним језиком преведено ова реченица значи да стање почиње из стања **Init** и прелази могу да се дешавају само према правилу **Next** или да стање променљиве `_count` остане непромењено. Ако би се десило да у систему може да се деси неки прелаз који није по правилу tla систем ће да провером да то утврди и пријави грешку.

Потпун преглед оператора се налази у поглављу 4.

2.2 Конфигурациони фајл

Конфигурациони фајл има једноставну структуру. У нашем малом примеру дефинише неке константе, дефинише шта је следеће стање и које је име правила прелаза на следеће могуће знање.

```
CONSTANTS
    greeting = "Hello"

INIT Init
NEXT Next
```

На крају поглавља четири има кратак преглед могућих опција за конфигурациони фајл.

Упозанавње са осталим могућим операторима и идејама tla спецификације ћемо урадити кроз примере. Треба напоменути да због специфичности временске компоненте оператора њихова правилна употреба није лака и да је најбоље упознати се са њима кроз мале практичне примере.

3 Неки основни примери

Овде дајемо неке примере из рачунарства који су довољно мали али илустративни да се покакжу неки концепти tla спецификације.

3.1 Прекидач за светло

Пример 3.1 (Прекидач за светло)**.** Прекидач за светло је класичан пример коначног аутомата са два стања: `off` и `on`. Једина акција је притискање прекидача (`Toggle`) која мења стање из `off` у `on` и обрнуто.

TLA+ модул (`LightSwitch.tla`)**:**

```
---- MODULE LightSwitch ----
VARIABLE light

Init == light = "off"

Toggle ==
    \/ /\ light = "off"
       /\ light' = "on"
    \/ /\ light = "on"
       /\ light' = "off"

Next == Toggle

Spec == Init /\ [][Next]_light

TypeInvariant == light \in {"off", "on"}
====
```

TLC конфигурациони фајл (`LightSwitch.cfg`)**:**

```
INIT Init
NEXT Next
INVARIANT TypeInvariant
```

Конфигурациони фајл говори TLC model checkeru:

- `INIT` — која формула описује почетна стања,
- `NEXT` — која формула описује прелазе стања,
- `INVARIANT` — које особине треба да важе у *свим* достижним стањима.

Овде први пут видимо нови концепт **INVARIANT** у конфигурационом фајлу. Он нам говори да неки услов треба да важи у свим стањима наше спецификације и према томе је инваријантан. Особина исправности је *светло је увек у неком од два дозвољена стања*, тј. `TypeInvariant` дефиниција важи у сваком достижном стању.

Такође обратимо пажњу на дефиницију

```
    TypeInvariant == light \in {"off", "on"}
```

овде можемо видети такође везу са математиком. Наиме ово је чиста формула теорије скупова која каже променљива `light` припада неком скупу могућих вредности. Могли смо све ово записати као $x \in \{p, q\}$, идеја програма би имала исти смисао.

Пример 3.2 (Прекидач за светло — булова варијанта)**.** Исти прекидач можемо моделовати и са буловом променљивом уместо стрингова. Вредност `TRUE` представља укључено светло, `FALSE` искључено. Акција `Toggle` тада постаје знатно краћа јер нема потребе за две гране.

TLA+ модул (`LightSwitchBool.tla`):

```
---- MODULE LightSwitchBool ----
VARIABLE light

Init == light = FALSE

Toggle == light' = ~light

Next == Toggle

Spec == Init /\ [][Next]_light

TypeInvariant == light \in BOOLEAN
====
```

Уочимо неколико ствари. `Toggle` је сада само `light' = ~light` негација тренутне вредности. `TypeInvariant` користи уграђени скуп `BOOLEAN` = {`TRUE`, `FALSE`} уместо ручно набројаног скупа стрингова. Обе варијанте су семантички еквивалентне; булова је компактнија, а стринг варијанта је читљивија када стања имају смислена имена. Приметимо да смо овде користили нови оператор `~` који нам представља логичку негацију.

3.2 Брава на вратима

Пример 3.3 (Брава на вратима)**.** Брава на вратима је пример са три стања: `locked`, `unlocked` и `open`. За разлику од прекидача, овде *нису све акције увек дозвољене* нпр. врата се не могу отворити ако су закључана.

Акције су:

- `Unlock` — откључавање (само из стања `locked`),
- `Lock` — закључавање (само из стања `unlocked`),
- `Open` — отварање (само из стања `unlocked`),
- `Close` — затварање (само из стања `open`).

TLA+ модул (`DoorLock.tla`):

```
---- MODULE DoorLock ----
VARIABLE door

Init == door = "locked"

Unlock == /\ door = "locked"
          /\ door' = "unlocked"

Lock   == /\ door = "unlocked"
```

```
          /\ door' = "locked"

Open   == /\ door = "unlocked"
          /\ door' = "open"

Close  == /\ door = "open"
          /\ door' = "unlocked"

Next == Unlock \/ Lock \/ Open \/ Close

Spec == Init /\ [][Next]_door

TypeInvariant == door \in {"locked", "unlocked", "open"}

CannotOpenWhenLocked == door = "locked" => door # "open"
====
```

TLC конфигурациони фајл (`DoorLock.cfg`)**:**

```
INIT Init
NEXT Next
INVARIANT TypeInvariant
INVARIANT CannotOpenWhenLocked
```

Кључна разлика у односу на прекидач је да свака акција садржи **услов изводљивости** (precondition) у првом конјункту. На пример, `Open` може да се изврши само ако је `door = "unlocked"` ако то не важи, акција је блокирана. TLC ће аутоматски проверити да у ниједном достижном стању `CannotOpenWhenLocked` не буде нарушена.

3.3 Узајамно искључење

Пример 3.4 (Узајамно искључење)**.** Узајамно искључење (*mutual exclusion*) је класичан проблем у теорији конкурентних система. Два процеса се такмиче за приступ критичној секцији, делу кода који сме да извршава само један процес у времену. Сваки процес може бити у једном од три стања: `idle` (не покушава улазак), `trying` (чека на улазак) и `critical` (у критичној секцији).

Овај пример је значајан јер уводи два кључна концепта:

- моделовање **вишеструких процеса** — стање система је функција која сваком процесу додељује његово тренутно стање,
- **safety особина** — гарантује да два процеса никад нису истовремено у критичној секцији.

TLA+ модул (`MutualExclusion.tla`)**:**

```
---- MODULE MutualExclusion ----
EXTENDS Naturals

CONSTANT N
ASSUME N \in Nat /\ N > 0

```

```
Procs == 1..N

VARIABLE state

States == {"idle", "trying", "critical"}

Init == state = [p \in Procs |-> "idle"]

TryEnter(p) ==
    /\ state[p] = "idle"
    /\ state' = [state EXCEPT ![p] = "trying"]

Enter(p) ==
    /\ state[p] = "trying"
    /\ \A q \in Procs : q # p => state[q] # "critical"
    /\ state' = [state EXCEPT ![p] = "critical"]

Exit(p) ==
    /\ state[p] = "critical"
    /\ state' = [state EXCEPT ![p] = "idle"]

Next == \E p \in Procs : TryEnter(p) \/ Enter(p) \/ Exit(p)

Spec == Init /\ [][Next]_state

TypeInvariant == \A p \in Procs : state[p] \in States

MutualExclusion == \A p, q \in Procs : p # q => ~(state[p] = "
   critical" /\ state[q] = "critical")
====
```

TLC конфигурациони фајл (`MutualExclusion.cfg`)**:**

```
CONSTANTS
    N = 2

INIT Init
NEXT Next
INVARIANT TypeInvariant
INVARIANT MutualExclusion
```

Разложимо сваки сегмент спецификације.

Параметар и почетно стање. `CONSTANT N` уводи број процеса као параметар чија вредност се задаје у конфиг фајлу. `ASSUME` гарантује да TLC не прихвата безначајне вредности попут `N = 0`. `Procs == 1..N` је скуп идентификатора процеса, нпр. за `N = 2` то је $\{1, 2\}$. Стање система је **функција** `state` дефинисана над скупом `Procs` израз `[p \in Procs |-> "idle"]` прави функцију која сваком процесу додељује `"idle"`.

Акција `TryEnter(p)`**.** Процес прелази из `idle` у `trying` пријављује намеру да уђе у критичну секцију. Нема додатних предуслова: сваки `idle` процес може у сваком тренутку покушати улазак. `[state EXCEPT ![p] = "trying"]` прави нову функцију идентичну `state` осим за процес `p`.

Акција `Enter(p)`**.** Процес прелази из `trying` у `critical`. Кључни предуслов је `\A q \in Procs : q # p => state[q] # "critical"`, процес `p` може ући само ако *ниједан* други процес тренутно није у критичној секцији. Ово је сама суштина узајамног искључења директно изражена у спецификацији. Важно је напоменути да ова спецификација **не описује механизам** (браве, семафоре) итд. она само описује *шта* смеме да се деси, а не *како* то постићи у имплементацији.

Акција `Exit(p)`**.** Процес напушта критичну секцију и враћа се у `idle`. Нема услова осим да процес мора бити у `critical`, систем не моделује шта се дешава унутар критичне секције.

Акција `Next`**.** `\E p \in Procs` означава да у сваком кораку TLC бира неки процес `p` и извршава једну од три акције за њега. Ово моделује **произвољно преплитање**, TLC ће истражити све могуће редоследе извршавања.

Особине. `TypeInvariant` проверава да `state[p]` увек остаје у скупу `States` за све процесе. `MutualExclusion` је главна safety особина: за свака два *различита* процеса `p` и `q` (`p # q`), не може се десити да су оба у `critical` истовремено. Симбол `~` означава логичку негацију, цела формула је импликација: ако су `p` и `q` различити, онда њихова стања не могу оба бити `"critical"`.

4 Напредни примери

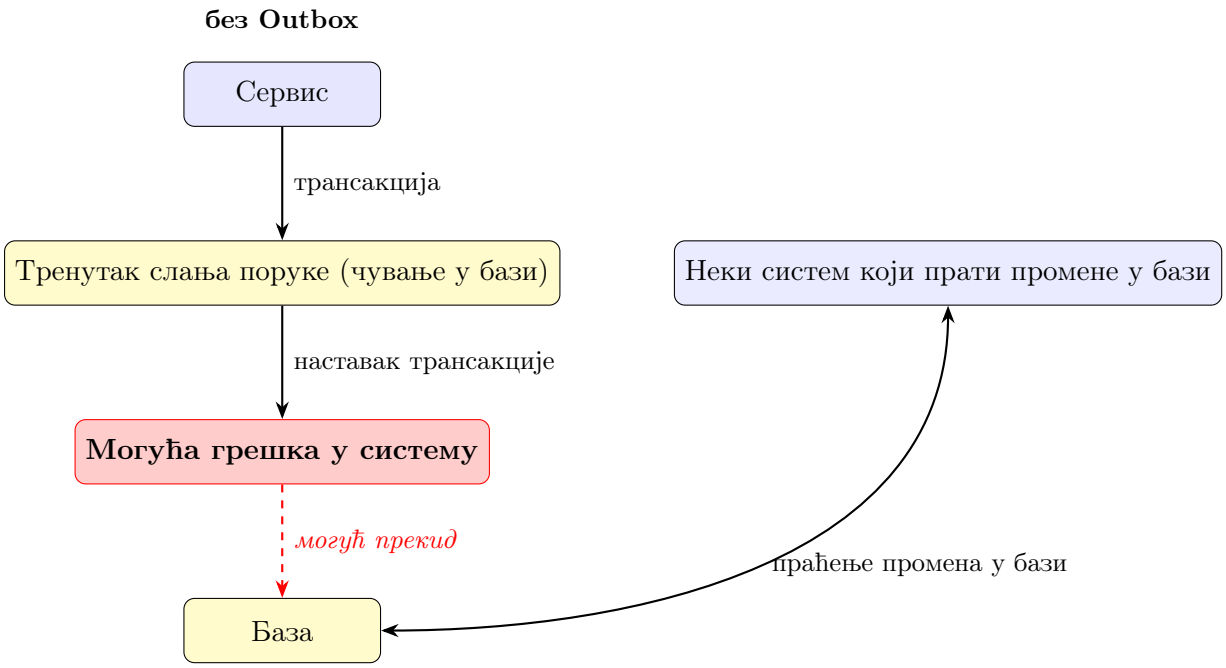
Ови примери претпостављају познавање основних концепата TLA+ и приказују реалистичније системе са вишеструким процесима, збиркама порука и нетривијалним особинама исправности.

4.1 Outbox pattern

Замислимо ситуацију да имамо сервис који комуницира са неким другим сервисима или спољним системима. Нека се та комуникација одвија преко неких порука које се могу слати на разне начине, конкретна технологија није битна. У тој ситуацији наш сервис поред слања поруке неком екстерном сервису, такође чува податке у бази података и желимо да будемо сигурни да ако се порука послала, податци такође сачувани у бази. Такође желимо да порука не буде послата ако се податци нису сачувалу у бази. Проблем који се јавља овде је да ако систем пошаље поруку и у међувремену пре се чувања у базу догоди нека грешка имамо велики проблем. Порука је послата, а сама целокупна логика сервиса није успела. Поруку наравно не можемо вратити „уназад". Ситуација је описана у псеудокоду и на слици.

Слика 8: Проблем комуникације са спољним системом и уписом у базу.

Једно од решења проблема је да поруку коју шаљемо спољном систему не шаљемо директно њему преко неког брокера порука, него и да саму поруку сачувамо директно у базу заједно са податцима.



Слика 9: Решење са чувањем поруке у базу заједно са податцима као део једне трансакције

Практично гледано оно сто се дешава написано у псеудо коду при оваквим понашању система.

Algorithm 1 Слање поруке спољном систему

```
// нека пословна логика...
припреми податке за чување
пошаљи поруку спољном систему(или сачувај у базу)
// нека додатна логика...
if дошло до грешке then
    прекини трансакцију!          ▷ порука је већ послата, не може се повући
    return
end if
// нека додатна логика...
сачувај податке у базу
return
```

Ако дође до прекида трансакције порука која је сачувана у базу би била такође обрисана и на тај начин не бисмо довели систем у неконзистентно стање. Конкретно, `outbox` табела у бази података могла би изгледати овако:

id	order	status	payload (чист бинарни запис или JSON)
0	o1	PENDING	`{type: "ORDER_CREATED", orderId: "o1"}`
1	o2	DELIVERED	`{type: "ORDER_CREATED", orderId: "o2"}`
2	o1	PENDING	`{type: "ORDER_UPDATED", orderId: "o1"}`

Пример 4.1 (Outbox pattern)**.** Систем моделујемо са четири променљиве:

- db — стање налога у бази ("NEW" или "CREATED"),
- outbox — скуп порука сачуваних у outbox табели,
- extSystem — скуп порука које је примио спољни систем,
- nextMsgId — бројач за јединствене идентификаторе порука.

Поруке са статусом PENDING чекају да их CDC (Change Data Capture) процес прочита и пошаље спољном систему. Након потврде статус прелази у DELIVERED и порука се може обрисати.

Постоје три акције. CreateOrder уписује промену у db и поруку у outbox у оквиру исте трансакције. DeliverMessage симулира засебан процес (нпр. CDC) који чита outbox и прослеђује поруку спољном систему. CleanupOutbox брише поруку из outbox тек кад је спољни систем потврди.

TLA+ модул (OutboxPattern.tla):

```
---- MODULE OutboxPattern ----
EXTENDS Naturals

VARIABLES db, outbox, extSystem, nextMsgId

Orders == {"o1", "o2"}

Init ==
    /\ db          = [o \in Orders |-> "NEW"]
    /\ outbox      = {}
    /\ extSystem   = {}
    /\ nextMsgId   = 0

CreateOrder(o) ==
    LET msg == [id |-> nextMsgId, order |-> o] IN
    /\ db[o] = "NEW"
    /\ db'         = [db EXCEPT ![o] = "CREATED"]
    /\ outbox'     = outbox \cup {msg}
    /\ nextMsgId' = nextMsgId + 1
    /\ UNCHANGED extSystem

DeliverMessage(msg) ==
    /\ msg \in outbox
    /\ msg \notin extSystem
    /\ extSystem' = extSystem \cup {msg}
    /\ UNCHANGED <<db, outbox, nextMsgId>>

CleanupOutbox(msg) ==
    /\ msg \in outbox
    /\ msg \in extSystem
    /\ outbox' = outbox \ {msg}
    /\ UNCHANGED <<db, extSystem, nextMsgId>>

Next ==
    \/ \E o \in Orders       : CreateOrder(o)
    \/ \E msg \in outbox     : DeliverMessage(msg)
```

```
    \/ \E msg \in outbox    : CleanupOutbox(msg)

Spec == Init /\ [][Next]_<<db, outbox, extSystem, nextMsgId>>

TypeInvariant ==
    /\ db \in [Orders -> {"NEW", "CREATED"}]
    /\ \A msg \in outbox    : msg.id \in Nat
    /\ \A msg \in extSystem : msg.id \in Nat

Consistency ==
    \A msg \in extSystem : msg \in outbox \/ msg.order \in Orders
====
```

TLC конфигурациони фајл (`OutboxPattern.cfg`):

```
INIT Init
NEXT Next
INVARIANT TypeInvariant
INVARIANT Consistency
```

Кључна особина коју TLC проверава је `Consistency`: свака порука коју прими спољни систем мора бити повезана са валидним налогом. Пошто `CreateOrder` уписује у `db` и `outbox` атомично, а порука стиже до спољног система тек преко `DeliverMessage`, ова особина увек важи.

Разложимо сваки сегмент спецификације. Овај део кода дефинише променљиве `db`, `outbox`, `extSystem` и `nextMsgId`. Дефинише константу `Orders`. Почетно стање је дефинисано као конјукција услова где су на почетку `outbox` и `extSystem` празни скупови. Прва порука почиње са јединственим иденфитикатором `nextMsgId` који је 0. Променљива `db` је дефинисана као константна функција којој је домен из скупа константе `Orders`, а кодомен само стринг `"NEW"`.

```
---- MODULE OutboxPattern ----
EXTENDS Naturals

VARIABLES db, outbox, extSystem, nextMsgId

Orders == {"o1", "o2"}

Init ==
    /\ db          = [o \in Orders |-> "NEW"]
    /\ outbox      = {}
    /\ extSystem   = {}
    /\ nextMsgId   = 0

...
====
```

Следећи блок дефинише акцију `CreateOrder(o)` која има параметар `o`, налог који се креира. `LET` дефинише локалну променљиву `msg` која је слог (*record*), што највише одговара структури у C-у или речнику у Python-у. Слог има два поља: `id` (јединствени идентификатор поруке) и `order` (налог на који се порука односи).

```
---
```

```
CreateOrder(o) ==
    LET msg == [id |-> nextMsgId, order |-> o] IN
    /\ db[o] = "NEW"
    /\ db'        = [db EXCEPT ![o] = "CREATED"]
    /\ outbox'    = outbox \cup {msg}
    /\ nextMsgId' = nextMsgId + 1
    /\ UNCHANGED extSystem

...
====
```

Тело акције је конјункција пет услова:

- `db[o] = "NEW"` — предуслов: акција може да се изврши само ако налог `o` још није обрађен,
- `db' = [db EXCEPT ![o] = "CREATED"]` — у следећем стању налог `o` добија статус `"CREATED"`; остали налози остају непромењени,
- `outbox' = outbox \cup {msg}` — порука `msg` се додаје у `outbox`; ово и претходна линија чине **атомичну трансакцију**,
- `nextMsgId' = nextMsgId + 1` — бројач се увећава да би следећа порука добила јединствен идентификатор,
- `UNCHANGED extSystem` — спољни систем не види ову промену; порука ће бити достављена тек акцијом `DeliverMessage`.

Акција `DeliverMessage(msg)` моделује CDC процес који чита `outbox` и прослеђује поруку спољном систему.

```
---
...
DeliverMessage(msg) ==
    /\ msg \in outbox
    /\ msg \notin extSystem
    /\ extSystem' = extSystem \cup {msg}
    /\ UNCHANGED <<db, outbox, nextMsgId>>

...
====
```

Тело акције је конјункција четири услова:

- `msg \in outbox` — предуслов: порука мора постојати у `outbox` табели; порука која није уписана у базу не може бити достављена,
- `msg \notin extSystem` — предуслов: порука још није достављена спољном систему; спречава дуплирано слање,
- `extSystem' = extSystem \cup {msg}` — у следећем стању порука се додаје у скуп примљених порука спољног система,
- `UNCHANGED «db, outbox, nextMsgId»` — база, `outbox` и бројач остају непромењени; брисање из `outbox` је одговорност акције `CleanupOutbox`.

Акција `CleanupOutbox(msg)` брише поруку из `outbox` табеле тек након што је спољни систем потврди.

```
---
CleanupOutbox(msg) ==
    /\ msg \in outbox
    /\ msg \in extSystem
    /\ outbox' = outbox \ {msg}
    /\ UNCHANGED <<db, extSystem, nextMsgId>>

...
====
```

Тело акције је конјункција четири услова:

- `msg \in outbox` — предуслов: порука мора и даље бити у `outbox` табели,
- `msg \in extSystem` — предуслов: порука је већ достављена спољном систему; без ове провере могли бисмо обрисати поруку пре него је послата,
- `outbox' = outbox \ {msg}` — у следећем стању порука се уклања из `outbox` табеле,
- `UNCHANGED «db, extSystem, nextMsgId»` — база, спољни систем и бројач остају непромењени.

Акција `Next` дефинише све могуће прелазе система као дисјункцију три акције. Оператор `\E` (постоји) означава да TLC бира конкретан налог `o` или поруку `msg` за коју је акција изводљива.

```
---
Next ==
    \/ \E o \in Orders       : CreateOrder(o)
    \/ \E msg \in outbox     : DeliverMessage(msg)
    \/ \E msg \in outbox     : CleanupOutbox(msg)

...
====
```

- прва грана бира неки налог `o` из скупа `Orders` и покушава да га креира,
- друга грана бира неку поруку из `outbox` и покушава да је достави,
- трећа грана бира неку поруку из `outbox` и покушава да је обрише ако је већ достављена.

TLC ће истражити све могуће комбинације избора у свим достижним стањима.

Формула `Spec` је потпуна темпорална спецификација система: `Init` описује почетно стање, а `[][Next]_vars` захтева да сваки прелаз или извршава неку акцију из `Next` или оставља све променљиве непромењеним (*stuttering step*).

```
---
Spec == Init /\ [][Next]_<<db, outbox, extSystem, nextMsgId>>

...
====
```

Суфикс `_«db, outbox, extSystem, nextMsgId»` наводи које променљиве чине стање система. Ако ниједна од ових променљивих не мења вредност, то је дозвољен корак, систем може да „стоји у месту”.

`TypeInvariant` је особина типске исправности која мора важити у *свим* достижним стањима.

```
---
TypeInvariant ==
    /\ db \in [Orders -> {"NEW", "CREATED"}]
    /\ \A msg \in outbox    : msg.id \in Nat
    /\ \A msg \in extSystem : msg.id \in Nat

...
====
```

Инваријанта је конјункција три услова:

- `db \in [Orders -> {"NEW", "CREATED"}]` — `db` је функција чији је домен тачно скуп `Orders`, а свака вредност је или `"NEW"` или `"CREATED"`; TLA+ запис `[A -> B]` означава скуп свих функција из `A` у `B`,
- `\A msg \in outbox : msg.id \in Nat` — сваки `id` у `outbox` је природан број,
- `\A msg \in extSystem : msg.id \in Nat` — исто важи за поруке које је примио спољни систем.

`Consistency` је главна safety особина система — гарантује да спољни систем никад не прими поруку која није повезана са легитимним налогом.

```
---
Consistency ==
    \A msg \in extSystem : msg \in outbox \/ msg.order \in Orders

...
====
```

Особина каже: за сваку поруку `msg` коју је примио спољни систем, мора важити барем један од два услова:

- `msg \in outbox` — порука и даље постоји у `outbox` табели (још није очишћена), или
- `msg.order \in Orders` — налог на који се порука односи постоји у скупу `Orders`.

Пошто акција `CreateOrder` уписује у `db` и `outbox` атомично, а `DeliverMessage` шаље само поруке које су у `outbox`-у, TLC ће потврдити да ова особина важи у свим достижним стањима.

5 Доказивање теорема у TLA+

TLA+ не служи само за проверу модела *(model checking)* тј. TLC проверава особине прегледом свих достижних стања, што је изводљиво само за коначне моделе са конкретним вредностима константи. За бесконачне системе или потпуно опште доказе користи се **TLAPS** (*TLA+ Proof System*), алат за формалне дедуктивне доказе директно у TLA+ синтакси [2].

TLAPS преводи доказе теорема на позадинске доказиваче теорема:

- **Zenon** је аутоматски доказивач за логику prvog реда,
- **Isabelle/TLA+** је интерактивни доказивач теорема,
- **SMT солвери** нпр. Z3 за аритметичке леме.

5.1 Структура доказа

Доказ у TLA+ је **хијерархијски** — разлаже се на нивое означене угластим заградама `<1>`, `<2>`, `<3>` итд. Сваки корак се означава бројем, нпр. `<1>1`, `<1>2`.

Кључне конструкције:

Конструкција	Значење
`THEOREM P`	исказ теореме коју треба доказати
`ASSUME H PROVE P`	доказ под претпоставком `H`
`BY facts`	закључи корак из наведених чињеница
`BY DEF Ime`	закључи разматрањем дефиниције `Ime`
`<1>n. QED`	завршни корак нивоа — обједињује претходне кораке
`PTL`	позива позадински доказивач за темпоралну логику

5.2 Пример: бројач са инваријантом

Моделујемо бројач који почиње од нуле и увећава се за један у сваком кораку. Желимо да формално докажемо да вредност бројача увек остаје природан број — особина која важи за *све* вредности бројача, без обзира на величину.

Пример 5.1 (Доказ инваријанте бројача)**. TLA+ модул (`CounterProof.tla`):**

```
---- MODULE CounterProof ----
EXTENDS Naturals, TLAPS

VARIABLE x

Init == x = 0
Next == x' = x + 1
Spec == Init /\ [][Next]_x

Inv == x \in Nat

THEOREM Spec => []Inv
<1>1. Init => Inv
      BY DEF Init, Inv
<1>2. Inv /\ [Next]_x => Inv'
```

```
      BY DEF Inv, Next
<1>3. QED
      BY <1>1, <1>2, PTL DEF Spec
====
```

Теорема каже: ако систем задовољава спецификацију `Spec`, онда `Inv` важи у *свим* стањима (темпорална формула `[]Inv`).

Доказ се одвија у три корака на нивоу `<1>`:

- `<1>1` — **базни случај**: ако је `Init` тачно, онда је `Inv` тачно. Разматрањем дефиниција: `Init` каже да је `x = 0`, а `Inv` каже да је `x \in Nat` — пошто је $0 \in \mathbb{N}$, следи тривијално.
- `<1>2` — **индуктивни корак**: ако `Inv` важи у тренутном стању и систем изврши корак `[Next]_x`, онда `Inv'` важи у следећем стању. Ако је `x \in Nat` и `x' = x + 1`, онда је `x' \in Nat` јер је скуп природних бројева затворен за сабирање.
- `<1>3. QED` — обједињује `<1>1` и `<1>2` позивањем `PTL` (*Propositional Temporal Logic*) доказивача, који из базног и индуктивног случаја аутоматски изводи `[]Inv`.

Ово је образац **индуктивног инваријанта** — најчешћа техника доказивања safety особина у TLA+:

1. Докажи да инваријанта важи у почетном стању (*initiation*).
2. Докажи да ако инваријанта важи пре прелаза, важи и после (*consecution*).
3. Из ова два корака TLAPS аутоматски изводи да инваријанта важи увек.

Пример 5.2 (Доказ безбедности прекидача за светло)**.** Враћамо се на пример прекидача за светло и формално доказујемо да `light` увек остаје у скупу дозвољених вредности — особина коју смо раније само проверавали TLC-ом.

TLA+ модул (`LightSwitchProof.tla`):

```
---- MODULE LightSwitchProof ----
EXTENDS TLAPS

VARIABLE light

Init == light = "off"

Toggle ==
    \/ /\ light = "off" /\ light' = "on"
    \/ /\ light = "on"  /\ light' = "off"

Next == Toggle

Spec == Init /\ [][Next]_light

TypeInvariant == light \in {"off", "on"}

THEOREM Spec => []TypeInvariant
```

```
<1>1. Init => TypeInvariant
      BY DEF Init, TypeInvariant
<1>2. TypeInvariant /\ [Next]_light => TypeInvariant'
  <2>1. ASSUME TypeInvariant, Toggle
        PROVE  TypeInvariant'
        BY <2>1 DEF TypeInvariant, Toggle
  <2>2. ASSUME TypeInvariant, UNCHANGED light
        PROVE  TypeInvariant'
        BY <2>2 DEF TypeInvariant
  <2>3. QED
        BY <2>1, <2>2 DEF Next
<1>3. QED
      BY <1>1, <1>2, PTL DEF Spec
====
```

Доказ следи исти образац индуктивног инваријанта, али са мање корака јер систем има само једну акцију.

- `<1>1` — у почетном стању `light = "off"`, па `light \in {"off", "on"}` тривијално важи.
- `<1>2` — индуктивни корак се рашчлањује на два подслучаја:
  - `<2>1`: акција `Toggle` мења `light` из `"off"` у `"on"` или обрнуто — у оба случаја вредност остаје у `{"off", "on"}`,
  - `<2>2`: ако `light` остаје непромењено, инваријанта тривијално важи.
- `<1>3. QED` — `PTL` обједињује базни и индуктивни случај у `[]TypeInvariant`.

Напомена. Model checking (TLC) и доказивање теорема (TLAPS) се **допуњују**: TLC је бржи и аутоматски проналази грешке у конкретним моделима, dok TLAPS даје потпун доказ за све могуће вредности параметара. Уобичајен радни ток је прво верификовати спецификацију са TLC-ом, а затим формално доказати кључне особине са TLAPS-ом.

6 Преглед TLA+ оператора

У овом поглављу дајемо кратак преглед свих основних TLA+ оператора и кључних речи.

6.1 Логички оператори

Оператор	Опис
`/\`	Логичко И (*and*)
`\/`	Логичко ИЛИ (*or*)
`~`	Логичка негација (*not*)
`=>`	Импликација: $A \Rightarrow B$
`<=>`	Еквиваленција: $A \Leftrightarrow B$

6.2 Релацијски и аритметички оператори

Оператор	Опис
`=`	Једнакост
`/=`	Неједнакост
`<`, `>`, `<=`, `>=`	Поређење бројева
`+`, `-`, `*`, `\div`, `%`	Аритметичке операције (захтева `EXTENDS Naturals/Integers/Reals`)

6.3 Оператори стања и прелаза

Оператор	Опис
`x'`	Вредност променљиве x у **следећем** стању
`UNCHANGED x`	Променљива x не мења вредност: $x' = x$
`UNCHANGED «x,y»`	Ниједна од наведених променљивих се не мења
`[][A]_x`	Темпорална формула: A важи у свим прелазима или x остаје непромењено
`<>P`	Темпорална формула: P ће **евентуално** постати тачно
`[]P`	Темпорална формула: P је **увек** тачно

6.4 Теоријско-скуповни оператори

Оператор	Опис
`\in`	Припадност скупу: $x \in S$
`\notin`	Неприпадност скупу: $x \notin S$
`\cup`	Унија скупова: $A \cup B$
`\cap`	Пресек скупова: $A \cap B$
`\`	Разлика скупова: $A \setminus B$
`SUBSET S`	Скуп свих подскупова скупа S
`UNION S`	Унија свих елемената скупа скупова S

6.5 Оператори над функцијама и торкама

Оператор	Опис
`f[x]`	Примена функције f на аргумент x
`[x \in S |-> e]`	Дефиниција функције: свако x из S се слика у e
`EXCEPT`	Ажурирање функције: `[f EXCEPT ![x] = v]`
`DOMAIN f`	Домен функције f
`«a, b, c»`	Торка са елементима a, b, c

6.6 Квантификатори и LET/IN

Оператор	Опис
`\A x \in S : P`	Универзална квантификација: P важи за све $x \in S$
`\E x \in S : P`	Егзистенцијална квантификација: постоји $x \in S$ за које важи P
`LET x == e IN ...`	Локална дефиниција: уводи x као скраћеницу за израз e

6.7 Кључне речи модула

Кључна реч	Опис
`MODULE`	Почетак модула
`EXTENDS`	Увоз другог модула (нпр. `Naturals`, `Sequences`, `TLC`)
`CONSTANTS`	Декларација константи чије вредности се задају у конфигурационом фајлу
`VARIABLES`	Декларација променљивих стања
`ASSUME`	Претпоставка о вредности константи
`INSTANCE`	Инстанцирање другог модула
`Init`	Предикат почетног стања
`Next`	Предикат прелаза (дефинише следеће стање)
`Spec`	Потпуна спецификација система

6.8 Оператор опсега

Оператор `a..b` (из модула `Naturals` или `Integers`) прави скуп свих целих бројева од a до b укључујући крајеве.

Израз	Опис
`a..b`	Скуп $\{a, a+1, \ldots, b\}$, нпр. `1..10` даје $\{1, 2, \ldots, 10\}$

6.9 Записи

Записи (*records*) су структуре са именованим пољима, слично структурама у C-у или речницима у Python-у.

Синтакса	Опис
`[f1 |-> v1, f2 |-> v2]`	Прављење записа са пољима `f1` и `f2`
`r.field`	Приступ пољу `field` записа `r`
`[f1 : S1, f2 : S2]`	Скуп свих записа где `f1` $\in$ `S1` и `f2` $\in$ `S2`
`[r EXCEPT !.field = v]`	Нови запис идентичан `r` осим поља `field` које добија вредност `v`

6.10 Секвенце (`EXTENDS Sequences`)

Секвенце су уређене листе вредности. Да би се користиле потребно је увести модул `Sequences`.

Оператор	Опис
`«a, b, c»`	Литерал секвенце са елементима a, b, c
`Len(s)`	Дужина секвенце `s`
`Head(s)`	Први елемент секвенце `s`
`Tail(s)`	Секвенца без првог елемента
`Append(s, e)`	Нова секвенца са елементом `e` додатим на крај
`s \o t`	Конкатенација секвенци `s` и `t`
`SubSeq(s, i, j)`	Подсеквенца од индекса `i` до `j`
`s[i]`	Елемент секвенце на позицији `i` (индексирање од 1)

6.11 Оператор `CHOOSE`

`CHOOSE` бира један одређени елемент скупа који задовољава дати предикат. Важно је напоменути да `CHOOSE` није случајан избор, увек враћа исти елемент за исти скуп и предикат. Ако ниједан елемент не задовољава предикат, резултат је недефинисан.

Синтакса	Опис
`CHOOSE x \in S : P(x)`	Бира елемент x из скупа S такав да важи $P(x)$
`CHOOSE x : P(x)`	Бира елемент (произвољног типа) такав да важи $P(x)$

6.12 Израз `CASE`

`CASE` је условна конструкција са вишеструким гранањем, алтернатива угнежђеним `IF/THEN/ELSE` изразима. Грана `OTHER` покрива све случајеве које претходне гране нису покриле.

Синтакса	Опис
`CASE p1 -> e1`	Ако је услов `p1` тачан, вредност је `e1`
`[] p2 -> e2`	Ако је услов `p2` тачан, вредност је `e2`
`[] OTHER -> e`	Подразумевана грана ако ниједан услов није тачан

6.13 Оператори правичности (Fairness)

Оператори правичности служе за описивање *liveness* особина, гарантују да систем неће заувек избегавати неку акцију ако је она изводљива.

Оператор	Опис
`WF_vars(A)`	**Слаба правичност** (*Weak Fairness*): ако акција A постане трајно изводљива, мора се извршити
`SF_vars(A)`	**Јака правичност** (*Strong Fairness*): ако акција A постаје изводљива бесконачно пута, мора се извршити

Слаба правичност спречава да систем „заглави"у стању у ком акција никада није извршена иако је увек могућа. Јака правичност је строжа, акција се мора извршити чак и ако се изводљивост повремено губи.

6.14 Оператор ENABLED

`ENABLED A` је предикат који је тачан ако и само ако постоји следеће стање у које се може прећи акцијом A из тренутног стања.

Синтакса	Опис
`ENABLED A`	Тачно ако акција A може да се изврши у тренутном стању

6.15 Модул FinSets (коначни скупови)

Модул `FinSets` пружа операторе за рад са коначним скуповима. Увози се са `EXTENDS FinSets`.

Оператор	Опис
`IsFiniteSet(S)`	Тачно ако је скуп S коначан
`Cardinality(S)`	Број елемената коначног скупа S

6.16 Конфигурациони фајл (.cfg)

Конфигурациони фајл говори TLC верификатору шта да провери и које вредности да додели константама. Сваки пројекат има засебан `.cfg` фајл са истим именом као и модул (нпр. `Counter.cfg`).

Кључна реч	Синтакса	Опис
`INIT`	`INIT Init`	Задаје предикат почетног стања
`NEXT`	`NEXT Next`	Задаје предикат прелаза
`SPEC`	`SPEC Spec`	Алтернатива за `INIT+NEXT`: задаје целу темпоралну формулу
`INVARIANT`	`INVARIANT Inv`	Инваријанта коју TLC проверава у сваком стању
`PROPERTY`	`PROPERTY Prop`	Темпорална особина (нпр. liveness) коју TLC проверава
`CONSTANTS`	`N = 3`	Додељује конкретну вредност константи из спецификације
`CONSTANTS` (скуп)	`N \in {1,2,3}`	TLC проверава за све вредности из датог скупа
`CONSTRAINT`	`CONSTRAINT StateConstr`	Ограничава простор стања на стања која задовољавају предикат
`ACTION_CONSTRAINT`	`ACTION_CONSTRAINT ActConstr`	Ограничава прелазе на оне који задовољавају предикат
`SYMMETRY`	`SYMMETRY Perm`	Задаје симетрију ради смањења простора стања
`VIEW`	`VIEW ViewExpr`	Дефинише шта TLC сматра јединственим стањем

Пример конфигурационог фајла за Counter модул:

```
INIT Init
NEXT Next
INVARIANT TypeInvariant
```

Пример са константама и провером особине:

```
CONSTANTS
    N = 5

INIT Init
NEXT Next
INVARIANT SafetyProp
PROPERTY LivenessProp
```

Напомена: `INVARIANT` се проверава у свим достижним стањима (safety), dok `PROPERTY` служи за темпоралне формуле попут $\Box\Diamond P$ (увек евентуално P).

Литература

[1] Leslie Lamport. Learning tla+. https://lamport.azurewebsites.net/tla/learning.html.

[2] TLAPS Development Team. Tlaps proof tactics — tla+ proof system documentation. https://proofs.tlapl.us/doc/web/content/Documentation/Tutorial/Tactics.html, 2024.

[3] Hillel Wayne. Learn tla+. https://learntla.com/index.html.